10
Graph Algorithms

The graph representations and traversal algorithms of Chapter 9 provide the basic building blocks for any computation on graph structures. In this chapter, we consider more advanced graph theory and algorithms.

Graph theory is the study of the properties of graph structures. It provides us with a language with which to talk about graphs. The key to solving many problems is identifying the fundamental graph-theoretic notion underlying the situation and then using classical algorithms to solve the resulting problem.

We begin with a overview of basic graph theory and follow with algorithms for finding important structures such as minimum spanning trees, shortest paths, and maximum flows.

10.1 Graph Theory

In this section, we provide a quick review of basic graph theory. Several excellent books on graph theory [PS03, Wes00] are available for more detailed information. We outline relevant algorithms which should be fairly simple to program given the machinery developed in the previous chapter.

10.1.1 Degree Properties

Graphs are made up of vertices and edges. The simplest property of a vertex is its *degree*, the number of edges incident upon it.

There are several important properties of vertex degrees. The sum of the vertex degrees in any undirected graph is twice the number of edges, since every edge contributes one to the degree of both adjacent vertices. A corollary to this is that every graph contains an even number of odd degree vertices. For directed graphs, the relevant degree condition is that the sum of the in-degrees of all vertices equals the sum of all out-degrees. The parity of vertex degrees has an important role in recognizing Eulerian cycles as discussed in Section 10.1.3.

Trees are undirected graphs which contain no cycles. Vertex degrees are important in the analysis of trees. A *leaf* of a tree is a vertex of degree 1. Every n-vertex tree contains $n-1$ edges, so all non-trivial trees contain at least two leaf vertices. Deleting a leaf leaves a smaller tree, trimming the tree instead of disconnecting it.

Rooted trees are directed graphs where every node except the root has in-degree 1. The leaves are the nodes with out-degree 0. *Binary trees* are rooted trees where every vertex has an out-degree of 0 or 2. At least half the vertices of all such binary trees must be leaves.

A *spanning tree* of a graph $G = (V, E)$ is a subset of edges $E' \subset E$ such that E' is a tree on V. Spanning trees exist for any connected graph; the parent relation encoding vertex discovery for either breadth-first or depth-first search suffices to construct one. The *minimum spanning tree* is an important property of weighted graphs, and discussed in Section 10.2.

10.1.2 Connectivity

A graph is *connected* if there is an undirected path between every pair of vertices. The existence of a spanning tree is sufficient to prove connectivity. A depth-first search-based connected components algorithm was presented in Section 9.4.2.

However, there are other notions of connectivity to be aware of. The *vertex (edge) connectivity* is the smallest number of vertices (edges) which must be deleted to disconnect the graph. The most interesting special case when there is a single weak link in the graph. A single vertex whose deletion disconnects the graph is called an *articulation vertex*; any graph without such a vertex is said to be *biconnected*. A single edge whose deletion disconnects the graph is called a *bridge*; any graph without such an edge is said to be *edge-biconnected*.

Testing for articulation vertices or bridges is easy via brute force. For each vertex/edge, delete it from the graph and test whether the resulting graph remains connected. Be sure to add that vertex/edge back before doing the next deletion!

In directed graphs we are often concerned with *strongly connected components*, that is, partitioning the graph into chunks such that there are directed paths between all pairs of vertices within a given chunk. Road networks should be strongly connected, or else there will be places you can drive to but not drive home from without violating one-way signs.

The following idea enables us to identify the strongly connected components in a graph. It is easy to find a directed cycle using depth-first search, since any back edge plus the down path in the DFS tree gives such a cycle. All vertices in this cycle must be in the same strongly connected component. Thus we can shrink (contract) the vertices

on this cycle down to a single vertex representing the component, and then repeat. This process terminates when no directed cycle remains, and each vertex represents one strongly connected component.

10.1.3 Cycles in Graphs

All non-tree connected graphs contain cycles. Particularly interesting are cycles which visit all the edges or vertices of the graph.

An *Eulerian cycle* is a tour which visits every edge of the graph exactly once. The children's puzzle of drawing a geometric figure without ever lifting your pencil from the paper is an instance of finding an Eulerian cycle (or path), where the vertices are the junctions in the drawing and the edges represent the lines to trace. A mailman's route is ideally an Eulerian cycle, so he can visit every street (edge) in the neighborhood once before returning home. Strictly speaking, Eulerian cycles are *circuits*, not cycles, because they may visit vertices more than once.

An undirected graph contains an Eulerian cycle if it is connected and every vertex is of even degree. Why? The circuit must enter and exit every vertex it encounters, implying that all degrees must be even. This idea also suggests a way to find an Eulerian cycle, by building it one cycle at a time. We can find a simple cycle in the graph using the DFS-based algorithm discussed in Section 9.4.1. Deleting the edges on this cycle leaves each vertex with even degree. Once we have partitioned the edges into edge-disjoint cycles, we can merge these cycles arbitrarily at common vertices to build an Eulerian cycle.

In the case of directed graphs, the relevant condition is that all vertices have the same in-degree as out-degree. Peeling off any cycle preserves this property, and thus Eulerian cycles of directed graphs can be built in the same manner. *Eulerian paths* are tours that visit every edge exactly once but might not end up where they started from. These allow exactly two vertices to have parity violations, one of which must be the starting node and the other the ending node.

A *Hamiltonian cycle* is a tour which visits every vertex of the graph exactly once. The traveling salesman problem asks for the shortest such tour on a weighted graph. An Eulerian cycle problem in $G = (V, E)$ can be reduced to a Hamiltonian cycle problem by constructing a graph $G' = (V', E')$ such that each vertex in V' represents an edge of E and there are edges in E' connecting all neighboring pair of edges from G.

Unfortunately, no efficient algorithm exists for solving Hamiltonian cycle problems. Thus you have two options on encountering one. If the graph is sufficiently small, it can be solved via backtracking. Each Hamiltonian cycle is described by a permutation of the vertices. We backtrack whenever there does not exist an edge from the latest vertex to an unvisited one. If the graph is too large for such an attack we must try to find an alternate formulation of the problem, perhaps as an Eulerian cycle problem on a different graph.

10.1.4 Planar Graphs

Planar graphs are those which can be drawn in the plane such that no two edges cross each other. Many of the graphs we commonly encounter are planar. Every tree is planar: can you describe how to construct a non-crossing drawing for a given tree? Every road network in the absence of concrete/steel bridges must be planar. The adjacency structure of convex polyhedra also yield planar graphs.

Planar graphs have several important properties. First, there is a tight relation between the number of vertices n, edges m, and faces f of any planar graph. *Euler's formula* states that $n - m + f = 2$. Trees contain $n - 1$ edges, so any planar drawing of a tree has exactly one face, namely, the outside face. Any embedding of a cube (8 vertices and 12 edges) must contain six faces, as those who have played dice can attest to.

Efficient algorithms exist for testing the planarity of a graph and finding non-crossing embeddings, but all are fairly complicated to implement. Euler's formula gives an easy way to prove that certain graphs are *not* planar, however. Every planar graph contains at most $3n-6$ edges for $n > 2$. This bound means that every planar graph must contain a vertex of degree at most 5, and deleting this vertex leaves a smaller planar graph with this same property. Testing whether a given drawing is a planar embedding is the same as testing whether any of a given set of line segments intersect, which will be discussed when we get to geometric algorithms.

10.2 Minimum Spanning Trees

A *spanning tree* of a graph $G = (V, E)$ is a subset of edges from E forming a tree connecting all vertices of V. For edge-weighted graphs, we are particularly interested in the *minimum spanning tree*, the spanning tree whose sum of edge weights is the smallest possible.

Minimum spanning trees are the answer whenever we need to connect a set of points (representing cities, junctions, or other locations) by the smallest amount of roadway, wire, or pipe. Any tree is the smallest possible connected graph in terms of number of edges, while the minimum spanning tree is the smallest connected graph in terms of edge weight.

The two main algorithms for computing minimum spanning trees are Kruskal's and Prim's, and both are covered in most any algorithms course. We will present Prim's algorithm here because we think it is simpler to program, and because it gives us Dijkstra's shortest path algorithm with very minimal changes.

First, we must generalize the graph data structure from Chapter 9 to support edge-weighted graphs. Each edge-entry previously contained only the other endpoint of the given edge. We must replace this by a record allowing us to annotate the edge with weights:

```
typedef struct {
        int v;                          /* neighboring vertex */
```

```
        int weight;                     /* edge weight */
} edge;

typedef struct {
        edge edges[MAXV+1][MAXDEGREE];  /* adjacency info */
        int degree[MAXV+1];             /* outdegree of each vertex */
        int nvertices;                  /* number of vertices in graph */
        int nedges;                     /* number of edges in graph */
} graph;
```

and update the various initialization and traversal algorithms appropriately. This is not a complicated task.

Prim's algorithm grows the minimum spanning tree in stages starting from a given vertex. At each iteration, we add one new vertex into the spanning tree. A greedy algorithm suffices for correctness: we always add the lowest-weight edge linking a vertex in the tree to a vertex on the outside.

The simplest implementation of this idea would assign each vertex a Boolean variable denoting whether it is already in the tree (the array `intree` in the code below), and then searches all edges at each iteration to find the minimum weight edge with exactly one `intree` vertex.

Our implementation is somewhat smarter. It keeps track of the cheapest edge from any tree vertex to every non-tree vertex in the graph. The cheapest edge over all remaining non-tree vertices gets added in each iteration. We must update the costs of getting to the non-tree vertices after each insertion. However, since the new vertex is the only change in the tree all possible edge-weight updates come from its outgoing edges:

```
prim(graph *g, int start)
{
        int i,j;                        /* counters */
        bool intree[MAXV];              /* is vertex in the tree yet? */
        int distance[MAXV];             /* vertex distance from start */
        int v;                          /* current vertex to process */
        int w;                          /* candidate next vertex */
        int weight;                     /* edge weight */
        int dist;                       /* shortest current distance */

        for (i=1; i<=g->nvertices; i++) {
                intree[i] = FALSE;
                distance[i] = MAXINT;
                parent[i] = -1;
        }

        distance[start] = 0;
        v = start;
```

```
        while (intree[v] == FALSE) {
            intree[v] = TRUE;
            for (i=0; i<g->degree[v]; i++) {
                w = g->edges[v][i].v;
                weight = g->edges[v][i].weight;
                if ((distance[w] > weight) && (intree[w]==FALSE)) {
                        distance[w] = weight;
                        parent[w] = v;
                }
            }

            v = 1;
            dist = MAXINT;
            for (i=2; i<=g->nvertices; i++)
                if ((intree[i]==FALSE) && (dist > distance[i])) {
                        dist = distance[i];
                        v = i;
                }
        }
}
```

The minimum spanning tree itself or its cost can be reconstructed in two different ways. The simplest method would be to augment this procedure with statements that print the edges as they are found or total the weight of all selected edges in a variable for later return. Alternately, since the tree topology is encoded by the `parent` array it plus the original graph tells you everything about the minimum spanning tree.

This minimum spanning tree algorithm has several interesting properties which help solve many closely related problems:

- *Maximum Spanning Trees* — Suppose we hire an evil telephone company to connect a bunch of houses together, and that this company will be paid a price proportional to the amount of wire they install. Naturally, they will want to build as expensive a spanning tree as possible. The *maximum spanning tree* of any graph can be found by simply negating the weights of all edges and running Prim's algorithm. The most negative tree in the negated graph is the maximum spanning tree in the original.

  Most graph algorithms do not adapt so nicely to negative numbers. Indeed, shortest path algorithms have trouble with negative numbers, and certainly do *not* generate the longest possible path using this technique.

- *Minimum Product Spanning Trees* — Suppose we want the spanning tree with the smallest product of edge weights, assuming all edge weights are positive. Since $\lg(a \cdot b) = \lg(a) + \lg(b)$, the minimum spanning tree on a graph where each edge weight is replaced with its logarithm gives the minimum product spanning tree.

- *Minimum Bottleneck Spanning Tree* — Sometimes we seek a spanning tree which minimizes the maximum edge weight over all such trees. In fact, the minimum spanning tree has this property. The proof follows directly from the correctness of Kruskal's algorithm.

  Such bottleneck spanning trees have interesting applications when the edge weights are interpreted as costs, capacities, or strengths. A less efficient but simpler way to solve such problems might be to delete all "heavy" edges from the graph and ask whether the result is still connected. These kind of tests can be done with simple BFS/DFS.

The minimum spanning tree of a graph is unique if all m edge weights in the graph are distinct. If not, the order in which Prim's algorithm breaks ties determines which minimum spanning tree the algorithm returns.

10.3 Shortest Paths

The problem of finding shortest paths in unweighted graphs was discussed in Section 9.3.1; breadth-first search does the job, and that is all she wrote. BFS does *not* suffice for finding shortest paths in weighted graphs, because the shortest weighted path from a to b does not necessarily contain the fewest number of edges. We all have favorite back-door driving/walking routes which use more turns than the simplest path, but which magically get us there faster by avoiding traffic, lights, etc.

In this section, we will implement two distinct algorithms for finding shortest paths in weighted graphs.

10.3.1 Dijkstra's Algorithm

Dijkstra's algorithm is the method of choice for finding the shortest path between two vertices in an edge- and/or vertex-weighted graph. Given a particular start vertex s, it finds the shortest path from s to every other vertex in the graph, including your desired destination t.

The basic idea is very similar to Prim's algorithm. In each iteration, we are going to add exactly one vertex to the tree of vertices for which we *know* the shortest path from s. Just as in Prim's, we will keep track of the best path seen to date for all vertices outside the tree, and insert them in order of increasing cost.

The difference between Dijkstra's and Prim's algorithms is how they rate the desirability of each outside vertex. In the minimum spanning tree problem, all we care about is the weight of the next potential tree edge. In shortest path, we want to include the outside vertex which is closest (in shortest-path distance) to the start. This is a function of both the new edge weight *and* the distance from the start of the tree-vertex it is adjacent to.

In fact, this change is very minor. Below we give an implementation of Dijkstra's algorithm based on changing exactly three lines from our Prim's implementation – one of which is simply the name of the function!

```
dijkstra(graph *g, int start)           /* WAS prim(g,start) */
{
        int i,j;                        /* counters */
        bool intree[MAXV];              /* is vertex in the tree yet? */
        int distance[MAXV];             /* vertex distance from start */
        int v;                          /* current vertex to process */
        int w;                          /* candidate next vertex */
        int weight;                     /* edge weight */
        int dist;                       /* shortest current distance */

        for (i=1; i<=g->nvertices; i++) {
                intree[i] = FALSE;
                distance[i] = MAXINT;
                parent[i] = -1;
        }

        distance[start] = 0;
        v = start;

        while (intree[v] == FALSE) {
            intree[v] = TRUE;
            for (i=0; i<g->degree[v]; i++) {
                 w = g->edges[v][i].v;
                 weight = g->edges[v][i].weight;
/* CHANGED */    if (distance[w] > (distance[v]+weight)) {
/* CHANGED */            distance[w] = distance[v]+weight;
                         parent[w] = v;
                 }
            }

            v = 1;
            dist = MAXINT;
            for (i=2; i<=g->nvertices; i++)
                 if ((intree[i]==FALSE) && (dist > distance[i])) {
                         dist = distance[i];
                         v = i;
                 }
        }
}
```

How do we use `dijkstra` to find the length of the shortest path from `start` to a given vertex t? This is exactly the value of `distance[t]`. How can we reconstruct the actual path? By following the backward `parent` pointers from t until we hit `start` (or `-1` if no such path exists), exactly as was done in the `find_path()` routine of Section 9.3.3.

Unlike Prim's, Dijkstra's algorithm only works on graphs without negative-cost edges. The reason is that midway through the execution we may encounter an edge with weight so negative that it changes the cheapest way to get from s to some other vertex already in the tree. Indeed, the most cost-effective way to get from your house to your next-door neighbor would be through the lobby of any bank offering you enough money to make the detour worthwhile.

Most applications do not feature negative-weight edges, making this discussion academic. Floyd's algorithm, discussed below, works correctly unless there are negative cost cycles, which grossly distort the shortest-path structure. Unless that bank limits its reward to one per customer, you could so benefit by making an infinite number of trips through the lobby that it would *never* pay to actually reach your destination!

10.3.2 All-Pairs Shortest Path

Many applications need to know the length of the shortest path between all pairs of vertices in a given graph. For example, suppose you want to find the "center" vertex, the one which minimizes the longest or average distance to all the other nodes. This might be the best place to start a new business. Or perhaps you need to know the *diameter* of the graph, the longest shortest-path distance between all pairs of vertices. This might correspond to the longest possible time it takes a letter or network packet to be delivered between two arbitrary destinations.

We could solve this problem by calling Dijkstra's algorithm from each of the n possible starting vertices. But Floyd's all-pairs shortest-path algorithm is an amazingly slick way to construct this distance matrix from the original weight matrix of the graph.

Floyd's algorithm is best employed on an adjacency matrix data structure, which is no extravagance since we have to store all n^2 pairwise distances anyway. Our `adjacency_matrix` type allocates space for the largest possible matrix, and keeps track of how many vertices are in the graph:

```
typedef struct {
        int weight[MAXV+1][MAXV+1];     /* adjacency/weight info */
        int nvertices;                  /* number of vertices in graph */
} adjacency_matrix;
```

A critical issue in any adjacency matrix implementation is how we denote the edges which are not present in the graph. For unweighted graphs, a common convention is that graph edges are denoted by 1 and non-edges by 0. This gives exactly the wrong interpretation if the numbers denote edge weights, for the non-edges get interpreted as a free ride between vertices. Instead, we should initialize each non-edge to `MAXINT`. This way we can both test whether it is present and automatically ignore it in shortest-path computations, since only real edges will be used unless `MAXINT` is less than the diameter of your graph.

```
initialize_adjacency_matrix(adjacency_matrix *g)
{
        int i,j;                        /* counters */
```

```
        g -> nvertices = 0;

        for (i=1; i<=MAXV; i++)
                for (j=1; j<=MAXV; j++)
                        g->weight[i][j] = MAXINT;
}

read_adjacency_matrix(adjacency_matrix *g, bool directed)
{
        int i;                          /* counter */
        int m;                          /* number of edges */
        int x,y,w;                      /* placeholder for edge/weight */

        initialize_adjacency_matrix(g);

        scanf("%d %d\n",&(g->nvertices),&m);

        for (i=1; i<=m; i++) {
                scanf("%d %d %d\n",&x,&y,&w);
                g->weight[x][y] = w;
                if (directed==FALSE) g->weight[y][x] = w;
        }
}
```

All this is fairly trivial. How do we find shortest paths in such a matrix? Floyd's algorithm starts by numbering the vertices of the graph from 1 to n, using these numbers not to label the vertices but to order them.

We will perform n iterations, where the kth iteration allows only the first k vertices as possible intermediate steps on the path between each pair of vertices x and y. When $k = 0$, we are allowed no intermediate vertices, so the only allowed paths consist of the original edges in the graph. Thus the initial all-pairs shortest-path matrix consists of the initial adjacency matrix. At each iteration, we allow a richer set of possible shortest paths. Allowing the kth vertex as a new possible intermediary helps only if there is a short path that goes through k, so

$$W[i,j]^k = \min(W[i,j]^{k-1}, W[i,k]^{k-1} + W[k,j]^{k-1})$$

The correctness of this is somewhat subtle, and we encourage you to convince yourself of it. But there is nothing subtle about how short and sweet the implementation is:

```
floyd(adjacency_matrix *g)
{
        int i,j;                        /* dimension counters */
        int k;                          /* intermediate vertex counter */
        int through_k;                  /* distance through vertex k */
```

```
        for (k=1; k<=g->nvertices; k++)
                for (i=1; i<=g->nvertices; i++)
                        for (j=1; j<=g->nvertices; j++) {
                                through_k = g->weight[i][k]+g->weight[k][j];
                                if (through_k < g->weight[i][j])
                                        g->weight[i][j] = through_k;
                        }
}
```

The output of Floyd's algorithm, as it is written, does not enable one to reconstruct the actual shortest path between any given pair of vertices. Use Dijkstra's algorithm if you care about the actual path. Note, however, that most all-pairs applications need only the resulting distance matrix. These jobs are what Floyd's algorithm was designed for.

Floyd's algorithm has another important application, that of computing the *transitive closure* of a directed graph. In analyzing a directed graph, we are often interested in which vertices are reachable from a given node.

For example, consider the *blackmail graph* defined on a set of n people, where there is a directed edge (i, j) if i has sensitive-enough private information on j so that i can get him to do whatever he wants. You wish to hire one of these n people to be your personal representative. Who has the most power in terms of blackmail potential?

A simplistic answer would be the vertex of highest degree, but an even better representative would be the person who has blackmail chains to the most other parties. Steve might only be able to blackmail Miguel directly, but if Miguel can blackmail everyone else then Steve is the man you want to hire.

The vertices reachable from any single node can be computed using using breadth-first or depth-first search. But the whole batch can be computed as an all-pairs shortest-path problem. If the shortest path from i to j remains `MAXINT` after running Floyd's algorithm, you can be sure there is no directed path from i to j. Any vertex pair of weight less than `MAXINT` must be reachable, both in the graph-theoretic and blackmail senses of the word.

10.4 Network Flows and Bipartite Matching

Any edge-weighted graph can be thought of as a network of pipes, where the weight of edge (i, j) measures the *capacity* of the pipe. Capacities can be thought of as a function of the cross-sectional area of the pipe – a wide pipe might be able to carry 10 units of flow in a given time where a narrower pipe might only be able to carry 5 units. For a given weighted graph G and two vertices s and t, the *network flow problem* asks for the maximum amount of flow which can be sent from s to t while respecting the maximum capacities of each pipe.

While the network flow problem is of independent interest, its primary importance is that of being able to solve other important graph problems. A classic example is bipartite matching. A *matching* in a graph $G = (V, E)$ is a subset of edges $E' \subset E$ such

that no two edges in E' share a vertex. Thus a matching pairs off certain vertices such that every vertex is in at most one such pair.

Graph G is *bipartite* or *two-colorable* if the vertices can be divided into two sets, say, L and R, such that all edges in G have one vertex in L and one vertex in R. Many naturally defined graphs are bipartite. For example, suppose certain vertices represent jobs to be done and the remaining vertices people who can potentially do them. The existence of edge (j, p) means that job j can potentially done by person p. Or let certain vertices represent boys and certain vertices girls, with edges representing compatible pairs. Matchings in these graphs have natural interpretations as job assignments or as marriages.

The largest possible bipartite matching can be found using network flow. Create a *source* node s which is connected to every vertex in L by an edge of weight 1. Create a *sink* node t which is connected to every vertex in R by an edge of weight 1. Assign every edge in the bipartite graph G a weight of 1. Now, the maximum possible flow from s to t defines the largest matching in G. Certainly we can find a flow as large as the matching, by taking exactly the matching edges and their source-to-sink connections. Further, there can be no greater possible flow. How can we ever hope to get more than one flow unit through any vertex?

The simplest network flow algorithm to implement is the Ford-Fulkerson augmenting path algorithm. For each edge, we will keep track of both the amount of flow currently going through the edge as well as its remaining *residual* capacity. Thus we must modify our `edge` structure to accommodate the extra fields:

```
typedef struct {
        int v;                          /* neighboring vertex */
        int capacity;                   /* capacity of edge */
        int flow;                       /* flow through edge */
        int residual;                   /* residual capacity of edge */
} edge;
```

We look for any path from source to sink that increases the total flow and use it to augment the total. We terminate on the optimal flow when no such *augmenting* path exists.

```
netflow(flow_graph *g, int source, int sink)
{
        int volume;             /* weight of the augmenting path */

        add_residual_edges(g);

        initialize_search(g);
        bfs(g,source);

        volume = path_volume(g, source, sink, parent);

        while (volume > 0) {
```

```
                augment_path(g,source,sink,parent,volume);
                initialize_search(g);
                bfs(g,source);
                volume = path_volume(g, source, sink, parent);
        }
}
```

Any augmenting path from source to sink increases the flow, so we can use `bfs` to find such a path in the appropriate graph. We are only allowed to walk along network edges which have remaining capacity or, in other words, positive residual flow. We use this predicate to help `bfs` distinguish between saturated and unsaturated edges:

```
bool valid_edge(edge e)
{
        if (e.residual > 0) return (TRUE);
        else return(FALSE);
}
```

Augmenting a path transfers the maximum possible volume from the residual capacity into positive flow. The amount we can move is limited by the path-edge with the smallest amount of residual capacity, just as the rate at which traffic can flow is limited by the most congested point.

```
int path_volume(flow_graph *g, int start, int end, int parents[])
{
        edge *e;                        /* edge in question */
        edge *find_edge();

        if (parents[end] == -1) return(0);

        e = find_edge(g,parents[end],end);

        if (start == parents[end])
                return(e->residual);
        else
                return( min(path_volume(g,start,parents[end],parents),
                            e->residual) );
}

edge *find_edge(flow_graph *g, int x, int y)
{
        int i;                          /* counter */

        for (i=0; i<g->degree[x]; i++)
                if (g->edges[x][i].v == y)
                        return( &g->edges[x][i] );
```

```
        return(NULL);
}
```

Sending an additional unit of flow along directed edge (i, j) reduces the residual capacity of edge (i, j) but *increases* the residual capacity of edge (j, i). Thus the act of augmenting a path requires looking up both forward and reverse edges for each link on the path.

```
augment_path(flow_graph *g, int start, int end, int parents[], int volume)
{
        edge *e;                          /* edge in question */
        edge *find_edge();

        if (start == end) return;

        e = find_edge(g,parents[end],end);
        e->flow += volume;
        e->residual -= volume;

        e = find_edge(g,end,parents[end]);
        e->residual += volume;

        augment_path(g,start,parents[end],parents,volume);
}
```

Initializing the flow graph requires creating directed flow edges (i, j) and (j, i) for each network edge $e = (i, j)$. The initial flows are all set to 0. The initial residual flow of (i, j) is set to the capacity of e, while the initial residual flow of (j, i) is set to 0.

Network flows are an advanced algorithmic technique, and recognizing whether a particular problem can be solved by network flow requires experience. We point the reader to books by Cook and Cunningham [CC97] and Ahuja, Magnanti, and Orlin [AMO93] for more detailed treatments of the subject.

10.5 Problems

10.5.1 Freckles

PC/UVa IDs: 111001/10034, **Popularity:** B, **Success rate:** average **Level:** 2

In an episode of the Dick Van Dyke show, little Richie connects the freckles on his Dad's back to form a picture of the Liberty Bell. Alas, one of the freckles turns out to be a scar, so his Ripley's engagement falls through.

Consider Dick's back to be a plane with freckles at various (x, y) locations. Your job is to tell Richie how to connect the dots so as to minimize the amount of ink used. Richie connects the dots by drawing straight lines between pairs, possibly lifting the pen between lines. When Richie is done there must be a sequence of connected lines from any freckle to any other freckle.

Input

The input begins with a single positive integer on a line by itself indicating the number of test cases, followed by a blank line.

The first line of each test case contains $0 < n \leq 100$, giving the number of freckles on Dick's back. For each freckle, a line follows; each following line contains two real numbers indicating the (x, y) coordinates of the freckle.

There is a blank line between each two consecutive test cases.

Output

For each test case, your program must print a single real number to two decimal places: the minimum total length of ink lines that can connect all the freckles. The output of each two consecutive cases must be separated by a blank line.

Sample Input

```
1

3
1.0 1.0
2.0 2.0
2.0 4.0
```

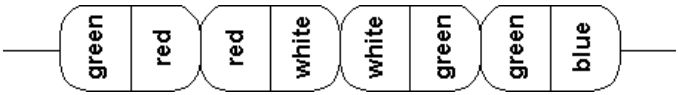
Sample Output

```
3.41
```

10.5.2 The Necklace

PC/UVa IDs: 111002/10054, **Popularity:** B, **Success rate:** low **Level:** 3

My little sister had a beautiful necklace made of colorful beads. Each two successive beads in the necklace shared a common color at their meeting point, as shown below:



But, alas! One day, the necklace tore and the beads were scattered all over the floor. My sister did her best to pick up all the beads, but she is not sure whether she found them all. Now she has come to me for help. She wants to know whether it is possible to make a necklace using all the beads she has in the same way that her original necklace was made. If so, how can the beads be so arranged?

Write a program to solve the problem.

Input

The first line of the input contains the integer T, giving the number of test cases. The first line of each test case contains an integer N ($5 \leq N \leq 1{,}000$) giving the number of beads my sister found. Each of the next N lines contains two integers describing the colors of a bead. Colors are represented by integers ranging from 1 to 50.

Output

For each test case, print the test case number as shown in the sample output. If reconstruction is impossible, print the sentence "`some beads may be lost`" on a line by itself. Otherwise, print N lines, each with a single bead description such that for $1 \leq i \leq N-1$, the second integer on line i must be the same as the first integer on line $i+1$. Additionally, the second integer on line N must be equal to the first integer on line 1. There may be many solutions, any one of which is acceptable.

Print a blank line between two successive test cases.

Sample Input

```
2
5
1 2
2 3
3 4
4 5
5 6
5
2 1
```

```
2 2
3 4
3 1
2 4
```

Sample Output

```
Case #1
some beads may be lost

Case #2
2 1
1 3
3 4
4 2
2 2
```

10.5.3 Fire Station

PC/UVa IDs: 111003/10278, **Popularity:** B, **Success rate:** low **Level:** 2

A city is served by a number of fire stations. Residents have complained that the distance between certain houses and the nearest station is too far, so a new station is to be built. You are to choose the location of the new station so as to reduce the distance to the nearest station from the houses of the poorest-served residents.

The city has up to 500 intersections, connected by road segments of various lengths. No more than 20 road segments intersect at a given intersection. The locations of houses and fire stations alike are considered to be at intersections. Furthermore, we assume that there is at least one house associated with every intersection. There may be more than one fire station per intersection.

Input

The input begins with a single line indicating the number of test cases, followed by a blank line. There will also be a blank line between each two consecutive inputs.

The first line of input contains two positive integers: the number of existing fire stations f ($f \leq 100$) and the number of intersections i ($i \leq 500$). Intersections are numbered from 1 to i consecutively. Then f lines follow, each containing the intersection number at which an existing fire station is found. A number of lines follow, each containing three positive integers: the number of an intersection, the number of a different intersection, and the length of the road segment connecting the intersections. All road segments are two-way (at least as far as fire engines are concerned), and there will exist a route between any pair of intersections.

Output

For each test case, output the lowest intersection number at which a new fire station can be built so as to minimize the maximum distance from any intersection to its nearest fire station. Separate the output of each two consecutive cases by a blank line.

Sample Input	*Sample Output*
1 1 6 2 1 2 10 2 3 10 3 4 10 4 5 10 5 6 10 6 1 10	5

10.5.4 Railroads

PC/UVa IDs: 111004/10039, **Popularity:** C, **Success rate:** average **Level:** 3

Tomorrow morning Jill must travel from Hamburg to Darmstadt to compete in the regional programming contest. Since she is afraid of arriving late and being excluded from the contest, she is looking for the train which gets her to Darmstadt as early as possible. However, she dislikes getting to the station too early, so if there are several schedules with the same arrival time then she will choose the one with the latest departure time.

Jill asks you to help her with her problem. You are given a set of railroad schedules from which you must compute the train with the earliest arrival time and the fastest connection from one location to another. Fortunately, Jill is very experienced in changing trains and can do this instantaneously, i.e., in zero time!

Input

The very first line of the input gives the number of scenarios. Each scenario consists of three parts. The first part lists the names of all cities connected by the railroads. It starts with a number $1 < C \leq 100$, followed by C lines containing city names. All names consist only of letters.

The second part describes all the trains running during a day. It starts with a number $T \leq 1{,}000$ followed by T train descriptions. Each of them consists of one line with a number $t_i \leq 100$ and then t_i more lines, each with a time and a city name, meaning that passengers can get on or off the train at that time at that city.

The final part consists of three lines: the first containing the earliest possible starting time, the second the name of the city where she starts, and the third with the destination city. The start and destination cities are always different.

Output

For each scenario print a line containing "`Scenario` i", where i is the scenario number starting from 1.

If a connection exists, print the two lines containing zero padded timestamps and locations as shown in the example. Use blanks to achieve the indentation. If no connection exists on the same day (i.e., arrival before midnight), print a line containing "`No connection`".

Print a blank line after each scenario.

Sample Input

```
2
3
Hamburg
Frankfurt
```

```
Darmstadt
3
2
0949 Hamburg
1006 Frankfurt
2
1325 Hamburg
1550 Darmstadt
2
1205 Frankfurt
1411 Darmstadt
0800
Hamburg
Darmstadt
2
Paris
Tokyo
1
2
0100 Paris
2300 Tokyo
0800
Paris
Tokyo
```

Sample Output

```
Scenario 1
Departure 0949 Hamburg
Arrival   1411 Darmstadt

Scenario 2
No connection
```

10.5.5 War

PC/UVa IDs: 111005/10158, **Popularity:** B, **Success rate:** average **Level:** 3

A war is being fought between two countries, A and B. As a loyal citizen of C, you decide to help your country by secretly attending the peace talks between A and B. There are n other people at the talks, but you do not know which person belongs to which country. You can see people talking to each other, and by observing their behavior during occasional one-to-one conversations you can guess if they are friends or enemies.

Your country needs to know whether certain pairs of people are from the same country, or whether they are enemies. You can expect to receive such questions from your government during the peace talks, and will have to give replies on the basis of your observations so far.

Now, more formally, consider a black box with the following operations:

`setFriends`(x,y)	shows that x and y are from the same country
`setEnemies`(x,y)	shows that x and y are from different countries
`areFriends`(x,y)	returns true if you are sure that x and y are friends
`areEnemies`(x,y)	returns true if you are sure that x and y are enemies

The first two operations should signal an error if they contradict your former knowledge. The two relations "friends" (denoted by $\sim$) and "enemies" (denoted by $*$) have the following properties:

$\sim$ is an equivalence relation: i.e.,

1. If $x \sim y$ and $y \sim z$, then $x \sim z$ (The friends of my friends are my friends as well.)
2. If $x \sim y$, then $y \sim x$ (Friendship is mutual.)
3. $x \sim x$ (Everyone is a friend of himself.)

$*$ is symmetric and irreflexive:

1. If $x * y$ then $y * x$ (Hatred is mutual.)
2. Not $x * x$ (Nobody is an enemy of himself.)
3. If $x * y$ and $y * z$ then $x \sim z$ (A common enemy makes two people friends.)
4. If $x \sim y$ and $y * z$ then $x * z$ (An enemy of a friend is an enemy.)

Operations `setFriends`(x,y) and `setEnemies`(x,y) must preserve these properties.

Input

The first line contains a single integer, n, the number of people. Each subsequent line contains a triple of integers, c x y, where c is the code of the operation,

$c = 1$,	`setFriends`
$c = 2$,	`setEnemies`
$c = 3$,	`areFriends`
$c = 4$,	`areEnemies`

and x and y are its parameters, integers in the range $[0, n)$ identifying two different people. The last line contains `0 0 0`.

All integers in the input file are separated by at least one space or line break. There are at most 10,000 people, but the number of operations is unconstrained.

Output

For every `areFriends` and `areEnemies` operation write "`0`" (meaning no) or "`1`" (meaning yes) to the output. For every `setFriends` or `setEnemies` operation which conflicts with previous knowledge, output a "`-1`" to the output; such an operation should produce no other effect and execution should continue. A successful `setFriends` or `setEnemies` gives no output.

All integers in the output file must be separated by one line break.

Sample Input

```
10
1 0 1
1 1 2
2 0 5
3 0 2
3 8 9
4 1 5
4 1 2
4 8 9
1 8 9
1 5 2
3 5 2
0 0 0
```

Sample Output

```
1
0
1
0
0
-1
0
```

10.5.6 Tourist Guide

PC/UVa IDs: 111006/10199, **Popularity:** B, **Success rate:** average **Level:** 3

Rio de Janeiro is a very beautiful city, but there are so many places to visit that sometimes you feel overwhelmed, Fortunately, your friend Bruno has promised to be your tour guide.

Unfortunately, Bruno is terrible driver. He has a lot of traffic fines to pay and is eager to avoid paying more. Therefore he wants to know where all the police cameras are located so he can drive more carefully when passing by them. These cameras are strategically distributed over the city, in locations that a driver must pass through in order to travel from one zone of the city to another. A location C will have a camera if and only if there are two city locations A and B such that all paths from A to B pass through a location C.

For instance, suppose that we have six locations (A, B, C, D, E, and F) with seven bidirectional routes $B - C$, $A - B$, $C - A$, $D - C$, $D - E$, $E - F$, and $F - C$. There must be a camera on C because to go from A to E you must pass through C. In this configuration, C is the only camera location.

Given a map of the city, help Bruno avoid further fines during your tour by writing a program to identify where all the cameras are.

Input

The input will consist of an arbitrary number of city maps, where each map begins with an integer N ($2 < N \leq 100$) denoting the total number of locations in the city. Then follow N different place names at one per line, where each place name will consist of least one and at most 30 lowercase letters. A non-negative integer R then follows, denoting the total routes of the city. The next R lines each describe a bidirectional route represented by the two places that the route connects.

Location names in route descriptions will always be valid, and there will be no route from one place to itself. You must read until $N = 0$, which should not be processed.

Output

For each city map you must print the following line:

```
City map #d: c camera(s) found
```

where d stands for the city map number (starting from 1) and c stands for the total number of cameras. Then should follow c lines with the location names of each camera in alphabetical order. Print a blank line between output sets.

Sample Input

```
6
sugarloaf
```

```
maracana
copacabana
ipanema
corcovado
lapa
7
ipanema copacabana
copacabana sugarloaf
ipanema sugarloaf
maracana lapa
sugarloaf maracana
corcovado sugarloaf
lapa corcovado
5
guanabarabay
downtown
botanicgarden
colombo
sambodromo
4
guanabarabay sambodromo
downtown sambodromo
sambodromo botanicgarden
colombo sambodromo
0
```

Sample Output

```
City map #1: 1 camera(s) found
sugarloaf

City map #2: 1 camera(s) found
sambodromo
```

10.5.7 The Grand Dinner

PC/UVa IDs: 111007/10249, **Popularity:** C, **Success rate:** high **Level:** 4

Each team participating in this year's ACM World Finals is expected to attend the grand banquet arranged for after the award ceremony. To maximize the amount of interaction between members of different teams, no two members of the same team will be allowed to sit at the same table.

Given the number of members on each team (including contestants, coaches, reserves, and guests) and the seating capacity of each table, determine whether it is possible for the teams to sit as described. If such an arrangement is possible, output one such seating assignment. If there are multiple possible arrangements, any one is acceptable.

Input

The input file may contain multiple test cases. The first line of each test case contains two integers, $1 \leq M \leq 70$ and $1 \leq N \leq 50$, denoting the number of teams and tables, respectively. The second line of each test case contains M integers, where the ith integer m_i indicates the number of members of team i. There are at most 100 members of any team. The third line contains N integers, where the jth integer n_j, $2 \leq n_j \leq 100$, indicates the seating capacity of table j.

A test case containing two zeros for M and N terminates the input.

Output

For each test case, print a line containing either 1 or 0, denoting whether there exists a valid seating arrangement of the team members. In case of a successful arrangement, print M additional lines where the ith line contains a table number (from 1 to N) for each of the members of team i.

Sample Input

```
4 5
4 5 3 5
3 5 2 6 4
4 5
4 5 3 5
3 5 2 6 3
0 0
```

Sample Output

```
1
1 2 4 5
1 2 3 4 5
2 4 5
1 2 3 4 5
0
```

10.5.8 The Problem With the Problem Setter

PC/UVa IDs: 111008/10092, **Popularity:** C, **Success rate:** average **Level:** 3

So many students are interested in participating in this year's regional programming contest that we have decided to arrange a screening test to identify the most promising candidates. This test may include as many as 100 problems drawn from as many as 20 categories. I have been assigned the job of setting problems for this test.

At first the job seemed to be very easy, since I was told that I would be given a pool of about 1,000 problems divided into appropriate categories. After getting the problems, however, I discovered that the original authors often wrote down multiple category-names in the category fields. Since no problem can used in more than one category and the number of problems needed for each category is fixed, assigning problems for the test is not so easy.

Input

The input file may contain multiple test cases, each of which begins with a line containing two integers, n_k and n_p, where n_k is the number of categories and n_p is the number of problems in the pool. There will be between 2 and 20 categories and at most 1,000 problems in the pool.

The second line contains n_k positive integers, where the ith integer specifies the number of problems to be included in category i ($1 \leq i \leq n_k$) of the test. You may assume that the sum of these n_k integers will never exceed 100. The jth ($1 \leq j \leq n_p$) of the next n_p lines contains the category information of the jth problem in the pool. Each such problem category specification starts with a positive integer specifying the number of categories in which this problem can be included, followed by the actual category numbers.

A test case containing two zeros for n_k and n_p terminates the input.

Output

For each test case, print a line reporting whether problems can be successfully selected from the pool under the given restrictions, with `1` for success and `0` for failure.

In case of successful selection, print n_k additional lines where the ith line contains the problem numbers that can be included in category i. Problem numbers are positive integers not greater then n_p and each two problem numbers must be separated by a single space. Any successful selection will be accepted.

Sample Input

```
3 15
3 3 4
2 1 2
1 3
```

```
1 3
1 3
1 3
3 1 2 3
2 2 3
2 1 3
1 2
1 2
2 1 2
2 1 3
2 1 2
1 1
3 1 2 3
3 15
7 3 4
2 1 2
1 1
1 2
1 2
1 3
3 1 2 3 2 2 3
2 2 3
1 2
1 2
2 2 3
2 2 3
2 1 2
1 1
3 1 2 3
0 0
```

Sample Output

```
1
8 11 12
1 6 7
2 3 4 5
0
```

10.6 Hints

10.1 Which problem from the chapter is Richie trying to solve?

10.2 Can this problem be modeled as a Hamiltonian or Eulerian cycle problem?

10.3 How can we use shortest-path information to help us position the station?

10.4 How can we model this as a shortest-path problem? What is the start node of our graph? How can we break ties in favor of trains leaving later in the day?

10.5 Can we propagate some of the implications of an observation through transitive closure?

10.6 What graph-theoretic concept defines the camera locations?

10.7 Does a greedy algorithm do the job, or must we use something like network flow?

10.8 Can this be modeled using network flow, or is there a more elementary approach?